The Gray-code CG fold

Narrowing the share-native fold without laddering it

2026-07-27

Assumes `docs/PRODUCT_SHARE_ENCODING` *and* `docs/SINGLE_CARRIER_CONSTRUCTION`. *Implements* `--prod-gray-fold`; the code is `ProdLedger::fold_cg_gray` and `emit_atom_onto` in `src/replace/gadgets.rs`. *Every number below is measured on the n=128 sliced sandwich unless it says otherwise.*

1 The problem

Under the product-share encoding a value is carried as

$$v \;=\; w_v \;\oplus\; x_1x_2 \;\oplus\; x_3x_4x_5 \;\oplus\; x_6x_7x_8,$$

the `[1,2,3,3]` plan: one carrier wire and three mask terms over the band. The CG for a source gate $a' = a \oplus bc$ must add $(w_b \oplus B)(w_c \oplus C)$ onto w_a alone, leaving every aux wire exactly as it found it.

The shipped fold expands that product into one gate per term of the cartesian product: with $1 + k_{\text{tot}} = 4$ atoms per operand, 16 fragments of width up to arity $\times$ max deg $= 6$.

Everything above two controls is invisible to the frozen store. Width-3 gates hit at 0.41% against $\sim$99% for narrow material, and widths 4–6 are absent from the store at any control cap or canonicalization budget — and at $n = 128$ that material is 56.2% of the gadget. So more than half the circuit is structurally beyond what phase A can ever re-encode.

The existing remedy is to ladder each wide fragment down with the Barenco double sweep (`--prod-max-width 2`), and it costs roughly 6.2$\times$ the fold. The reason is pure redundancy: every one of the nine mask-bearing fragments re-derives the same mask products from scratch.

2 The construction

Write $S_w = L_w \oplus M_w$ for each operand, splitting its atoms by width: L_w collects the width-≤ 1 atoms (the carrier literal, and the ledger's constant atom when the control is negative or the value carries a constant), and M_w is the sum of the mask terms. Borrow two wires u, z and toggle them around the Gray cycle over the state *(u holds M_b, z holds M_c)*:

$$A = (0,0) \xrightarrow{\text{gather } b} B = (1,0) \xrightarrow{\text{gather } c} C = (1,1) \xrightarrow{\text{strip } b} D = (0,1) \xrightarrow{\text{strip } c} A.$$

Let U_1, U_2 be u's value in the two columns and Z_1, Z_2 be z's in the two rows. Then

$$A_u \;=\; U_1 \oplus U_2 \;=\; M_b, \qquad A_z \;=\; Z_1 \oplus Z_2 \;=\; M_c,$$

because the borrows' unknown incoming values u_0, z_0 appear in both readings and cancel — the same cancellation the ladder's double sweep already relies on. Now emit

- $L_b \times L_c$, once per pair, at any phase;
- $L_b \times A_z$ once in each z-row, and $A_u \times L_c$ once in each u-column;
- $A_u \times A_z$ once at *every* one of the four phases.

The sum is $(L_b \oplus A_u)(L_c \oplus A_z) = S_b S_c$, which is the CG, and every one of those gates has at most two controls by construction — the operands of each are one width-≤ 1 atom and/or one accumulator literal.

Mask terms of degree 3 are gathered by a four-gate sandwich over a third dirty borrow v: accumulator gate, rung, accumulator gate, rung, so v is visited an even number of times and is restored. At `[1,2,3,3]` a block is 45 fragments against the wide fold's 16 — but *none of them is wide*.

The implementation is generic over arity, atom count and mask degree; it reads the atom lists `fold_cg` already builds rather than assuming a plan.

3 Why the accumulators must stay dirty

$w_b \oplus M_b$ *is* b. An accumulator that started clean would hold the full mask sum next to a live carrier, putting the operand one XOR away from a wire pair — the same use-point re-exposure that sank the deferred-mask peek. Borrowed, the wire holds $u_0 \oplus M_b$, and u_0 is off the wire set for the whole dirty window.

This is audited rather than argued. Over all 46 prefixes of a `[1,2,3,3]` block, computing the exact maximum correlation over *all* affine predictors in the live wire values — Walsh transform per connected component of the residual, with mask bits restricted to variables still live, no capped components — gives

$$\max_{\text{prefix}} \max_{\text{affine } \ell} \left|\mathrm{corr}(\ell, s)\right| \;=\; 0.28125 \;=\; \tfrac{1}{2}\left(\tfrac{3}{4}\right)^2 \qquad \text{for } s \in \{a, b, c, a'\},$$

which is exactly the encoding's own steady-state piling-up bound for `[2,3,3]`. The block's interior gives an affine adversary nothing its endpoints do not. Two further probes agree: no secret enters the GF(2) span of the wires and all their pairwise products at any prefix, and a beam search over quadratic mirror features peaks at the $(3/4)^2 = 0.5625$ design bound.

The instrument is sensitive to the failure it guards. Run against a CLEAN-accumulator variant of the same schedule, it recovers b at correlation 1.0 and flags 35 prefixes on the span test.

Admissibility, in the spirit. `u += x1*x2` does park a whole degree-2 atom's literal pair in one gate's support — as the shipped fold's carrier×mask2 fragments already do. It sits on top of u_0, which is what the measurement above measures. Degree-3 atoms are never named whole by any single gate, which is strictly better than the width-4–6 fragments this replaces. Hiding the pair as well would cost about +12 gates per block via a CNOT-copy detour, and is deliberately not implemented.

4 Two silent failure modes

Residual constants change the function, not the leak. `emit_g57_form` leaves a complement on its target, so a gather actually lands $M_b \oplus \delta$. Untreated, the four-phase sum becomes $(M_b \oplus \delta)(M_c \oplus \varepsilon)$: the block silently acquires $\delta M_c \oplus \varepsilon M_b$ and computes the *wrong function*. It is absorbed for free by toggling the operand's constant atom, since $L'_w = L_w \oplus \delta$ restores $L'_w \oplus A_w = S_w$ — the same ledger mechanism the wide fold already uses for a negative control.

A random pivot corrupts the borrow. `emit_narrow_fragment` chooses its ladder pivot *at random* among equally-scored candidates. Routing a gather and its strip through it independently therefore leaves *different* residuals, and the accumulator comes back off by one: not a wrong constant but a corrupted wire, permanently. `emit_atom_onto` takes the pivot as a parameter and the caller draws it once per atom, reusing it across both passes. Spellings still vary between the passes, because every spelling of one function carries the same constant.

Both were caught by the exactness test below, and neither is visible in a sampled check — the first only shows up on inputs where the polluting term fires, the second only when the two passes happen to draw different pivots.

5 Cost, and what it buys

Measured on the $n = 128$ sliced sandwich ($|C| = |D| = 3000$, seed 1, production preset otherwise). All three runs share the same sandwich and the same source C and differ only in the fold; all three report `verify PASSED (200 samples, low 256 wires)`.

	wide fold	gray fold	gray + `--prod-ladder-cap 3`
gadget gates	339,786	808,618 (2.38×)	1,021,244 (3.01×)
fold fossils (>2 ctrl)	153,421	**0**	**0**
gray blocks	0	7,209	7,209
store-reachable	**31.55%**	**95.47%**	**99.87%**
policy-blocked (>2 ctrl)	54.96%	4.28%	0.08%
store-blocked	13.49%	0.25%	0.04%
CNOT reachable	70.6%	99.8%	—
g57 reachable	69.5%	99.8%	—

Read reachability, not match rate: the two move in opposite directions, and reachability is the per-gate question of whether *any* window containing this gate resolves to something the store holds. 95.47% is above the previous best on record (82.40%, at `--prod-ladder-cap 3` alone, which cost 2.2× for 5.7× the reachable material and peaked there).

The residual 4.28% of wide gates under the plain gray fold is *not* the fold, which emits none. They are mask-slot emissions: a degree-3 mask term is a 3-control conjunction every time it is injected, re-sourced, retired or stripped (`emit_slot`), and that path honours `--prod-ladder-cap` alone. Adding `--prod-ladder-cap 3` ladders those too.

That is worth stating carefully, because it looks like it contradicts an earlier finding. Laddering was measured to *peak* at cap 4 and decline after, since deep multi-rung ladders manufacture material the store cannot reach. That finding is intact, and it is precisely why the Gray fold does not ladder

anything: it *removes* the wide fragments instead of spelling them out, which leaves cap 3 doing only what it was measured to be good at — single-rung, width-3 slot emissions.

Wire count is unchanged. Accumulators and sandwich helpers are borrowed from the existing carrier roles and restored, so the gadget's width is exactly what it was.

6 The duplicate-gate question

The Gray structure emits the same function on the same wires several times by construction: $A_u \times A_z$ at four phases, and each accumulator gate and rung twice per pass and twice again in the strip. That is the shape the `sort | uniq -c` census once used to locate every ladder, so it was measured.

It must be read against a null. A circuit with 2.4× the gates drawn from the same shape space collides far more by chance — at 379k CNOTs over 512 wires the pigeonhole floor alone is $\sim$76%. The null below keeps every gate's shape (comp, arity, polarity pattern) and the per-shape counts, and resamples only the wire labels.

$n = 128$	in exact pairs: obs	null	excess
wide fold	14.9%	6.0%	+8.9 pts
gray fold	45.1%	16.0%	+29.0 pts
gray + cap 3	30.4%	13.9%	+16.5 pts

So a real excess remains, and two attempts to remove it were measured and **rejected**, both recorded at their call sites so they are not re-tried:

1. Forcing the accumulator gate same-polarity. This is free *algebraically* — the sweep contributes $\lambda\,(f(h_0) \oplus f(h_0 \oplus R))$ for whichever literal f of the helper is used, and negating it adds 1 to both readings, which cancels — so the helper literal's polarity may be matched to λ's at will. It costs +5.7% gates and improves the census by nothing.
2. Steering the pivot toward a same-polarity rung (always available among three literals, by pigeonhole). Same outcome.

Both fail for the same reason: the four equal-size spellings of a same-polarity conjunction are only *two gate multisets reordered*, and the duplicate census — like `commuting_shuffle`, which runs afterwards anyway — is order-blind. Removing the excess would need *different wires* per emission, which the cancellation identity forbids.

What makes this acceptable is the reachability result. At 95.5% (99.9% with cap 3), phase A can re-encode essentially every gate in the circuit, where the wide fold left 55% of it permanently out of reach. The duplicate structure is a property of the *raw* gadget, and fmix is now in a position to churn it. The honest place to check that is a post-fmix census; it is an open measurement, not a claim made here.

7 Scope and fallbacks

The fold declines, and the block falls through to the odometer with laddering forced on, when the source gate is not arity 2 (an X or CNOT source: nothing to amortize), when an operand has no mask terms (the plain expansion is already at most $1 + \max \deg$ wide), when a mask term has

degree > 3 (a helper chain rather than one sandwich; the production plan is `[1,2,3,3]` and the generalization is unaudited), or when the carrier pool leaves no room to borrow two accumulators and a helper, which happens only at toy widths. At $n = 128$, 7,209 of 7,919 source gates take the Gray path; the rest are the arity-≤ 1 sources.

Borrows are drawn by **role** through `pairs/loc`, never by wire index. The home index range stops describing the carrier set the moment `--prod-roll` is on, and borrowing by index leaves a static index-shaped trace that rolling cannot average away — measured write-count AUC $0.518 \to 0.875$ when that was last got wrong.

8 Tests

`prod_gray_fold_is_share_native_and_two_control` Over the *whole* input domain and six source-gate shapes (g57, mixed- and equal-polarity CCNOTs, CNOT, X): the target value's decode transitions by exactly the virtual gate, every other value is untouched, every borrowed accumulator and sandwich helper is restored, and no emitted gate exceeds two controls. This is what catches a residual-parity slip, which is otherwise a silent wrong-function bug. It also asserts that the arity-2 sources actually took the Gray path, so it cannot silently degrade into a re-test of the odometer.

`prod_gray_fold_keeps_the_accumulators_dirty` The borrows are drawn from the carrier roles, are distinct from the target and from every operand literal, and each is written an even number of times so its incoming junk survives to cancel.

The library suite is 189/189 with these added. One test in the tree, `poly_canon_failure_case`, fails identically before and after this change and is unrelated.

The exposure audit is not a unit test — it is the standing script set in `cg_audit/` (`audit_cg_narrow.py` for one member, `audit_par.py` for the parallel sweep, `audit_rand_par.py` over the phase-assignment $\times$ pivot family the implementation actually draws from, and `dup_census.py` for the duplicate null). Re-run it when the schedule or the mask plan changes.